

LingShu Architecture Standard

灵枢是通用 **agent** 能力中枢，不是单一开发工具。代码结构必须服务于长期扩展：实时语音、视觉解析、外部 **agent**、多个模型网关、任务记忆和执行审计都应能独立演进。

Core Principle

灵枢主线程只做判断、记忆恢复、能力编排、权限裁决、验收和统一回复。具体工作由内部或外部能力节点完成。

任何新能力都必须先回答三个问题：

1. 它属于用户界面、应用编排、领域规则、基础设施适配，还是外部服务？
2. 它是否会让主线程直接承担具体执行工作？
3. 它产生的过程和产物是否能被任务执行记录追溯？

Layer Boundaries

App

路径：Sources/LingShuMac.swift

只允许放 **App** 启动、窗口初始化和根视图挂载。不得放业务模型、模型调用、**agent** 调度或大块 **UI** 组件。

Domain

路径：Sources/Domain/

放稳定领域对象和枚举，例如消息、**agent** 状态、任务阶段、记忆记录、模型配置描述。**Domain** 不得调用网络、文件系统、Codex CLI 或 SwiftUI 页面逻辑。

允许依赖：**Foundation**，必要时允许轻量 **UI** 值对象，例如 **Color**。

禁止依赖：具体模型 SDK、Codex、外部 **agent** HTTP 客户端、视图状态。

State

路径：Sources/State/

放界面可观察状态和主线程编排状态。**State** 可以组合服务，但不应直接实现底层模型协议、文件持久化细节或外部 **agent** 协议。

规则：

1. 主线程状态必须常驻，负责心跳和最近会话恢复。
2. 每条用户消息都对应一个任务执行记录。
3. 主线程先判断是否直接回答，再决定是否创建任务线程或调用 **agent**。
4. 普通问答也要经过记忆检索，但不一定创建子线程。
5. 本地判定为普通对话的消息走直答快路（**LingShuState+DirectChat.swift**：无 JSON 包装、灵枢人格流式直答，实测首正文 ~1.6s）；交付型诉求才进入路由编排。误判由直答模型的「【任务】」升级标记兜底——命中标记立即切回完整调度流程。

6. 路由、直答、执行三条流式链路都必须带延迟探针（首响/首正文/完成进调用链），体验回归以真实数字为准。
7. 情境上下文（**LingShuSituationContext**）常驻注入提示词：本机时间与时段、连续使用时长、说话人声线画像、视觉环境摘要、后台任务进展。情境如何影响回应（深夜提醒休息、对环境打趣、按声线调语气）由模型判断，禁止在代码里写死"条件→台词"策略。
8. 主线程只分配与汇报，不亲自做工：任务由协同管线执行（见 **Task Journal Standard**），完成后由灵枢主动发起结构化汇报（交付/过程/验收结论），不等用户来问。
9. 创建新任务上下文前必须过资源准入（**LingShuTaskAdmissionPolicy**：在跑管线数 × CPU 负载 × 内存余量），不适合立刻执行就入队并把理由告诉用户；环境合适后队列自动推进。
10. 任务执行期间主线程保持可对话：非任务消息走并发直答（独立任务句柄，不触碰执行状态机），灵枢可结合后台任务进展像朋友一样互动和给建议。

Application Services

路径：Sources/Services/

放语音、视觉、任务生命周期、执行记录聚合等应用级服务。服务可以被 **State** 调用，但服务之间不得形成循环依赖。

新增实时语音能力时，入口应放在 **Services/Voice** 或现有 **VoiceIOManager** 的后续拆分文件中。新增视觉解析能力时，入口应放在 **Services/Vision**，模型适配走 **Infrastructure/ModelGateway**。

实时感知能力必须分三层：

1. 设备采集层：麦克风、音箱、摄像头，只负责采集和播放，不直接决定业务流程。
2. 本地解析层：**Speech**、**Vision**、**TTS** 等系统能力，用于低延迟兜底和权限内观察。
3. 实时感知网关：统一封装音频包、视频帧、转写文本、视觉摘要和语音输出事件，按路由转发给本地解析、实时大模型或外部适配器。

设备层不得硬编码任何模型供应商。新增 **GPT**、**DeepSeek**、**MiniMax**、**Claude**、自托管视觉/音频模型时，必须通过 **LingShuRealtimePerceptionGateway** 或其后续拆分出来的 **provider** 注册，不得直接改麦克风或摄像头服务。

灵枢动画属于“身体表现层”，不是新的决策层，也不得占用对话主区。

LingShuDigitalHumanExpression / LingShuDigitalHumanDirective 描述光球的聆听、发声、思考、执行、警戒、确认、演示等表现态；**LingShuState+DigitalHuman** 只负责把大脑指令或真实运行状态转成表现快照；**LingShuDigitalHumanViews** 只消费快照，把左上角灵枢本体头像渲染成 **Jarvis** 风格光球。发声特效必须由真实输出电平驱动，而不是由“正在请求 **TTS / isSpeaking**”这种粗状态驱动；网络卡顿、缓冲饿住或无实际音频时，嘴部/声纹表现必须同步降为静默。后续接 **Live2D**、**VRM**、**MetaHuman** 或 **3D** 形象时，只能替换/新增表现 **view** 和渲染 **runtime**，不得让动画模块判断业务流程、创建任务或调用模型。

认主流程属于实时感知网关后的身份锁能力，而不是麦克风、摄像头或某个模型供应商的私有逻辑。开启认主后，灵枢必须同时采集面容和声线样本，形成可替换的主人身份档案；身份锁开启后，触发词命中也必须先通过面容和声线联合确认，才能进入实时对话或自动执行任务。当前本地实现只作为可演示的轻量特征锁，后续 **H100**、云端声纹、人脸识别或多模态模型只能替换特征提取和校验适配层，不能改动主线程对话流程。

默认身份档案位置：~/Library/Application Support/LingShu/Identity/owner-profile.json。档案只保存面容/声线特征向量和阈值，不保存原始音频或视频帧。

语音提交必须经过声线寻址闸门（**LingShuVoiceAddressingGate**，纯函数可插拔）：以绑定的主人声线为最高优先——身份锁开启时逐句核验声线，不匹配一律按环境音忽略（点名也不放行）；多人对话环境（声线画像基频双簇判定）未点名灵枢时，只有对话延续窗口内才响应，否则判定为人际交谈不插话。被忽略的语句进调用链可审计。**VAD** 阈值由 **LingShuAdaptiveVAD** 自适应：跟踪环境噪声底，嘈杂环境自动抬高说话/收口/打断门槛。

Infrastructure

路径：Sources/Infrastructure/

放外部世界适配，包括 Codex CLI、OpenAI/DeepSeek/Claude/MiniMax 等模型网关、外部 agent 协议、文件系统仓储、网络客户端。

规则：

1. **Infrastructure** 不决定产品流程，只提供能力。
2. 模型供应商差异必须封装在适配层，不得散落在 **UI** 或 **State** 中。
3. 长连接、流式输出、心跳、自恢复和重连策略属于模型网关或远端会话池，不属于视图。
4. 内部诊断日志不得直接进入用户对话，只能进入执行记录或调试窗。
5. 实时音视频模型如果需要 **WebSocket**、**SSE**、**gRPC** 或厂商 **SDK**，应在 **Infrastructure** 增加 **provider/adapter**，并对外暴露统一的实时感知事件合同。
6. 新增 **Kimi**、**DeepSeek**、**Qwen**、**MiniMax**、本地 **vLLM** 等主模型通道时，只能通过模型网关适配层声明接口、协议和能力；不得让业务流程依赖具体模型名称。
7. 模型回复方言必须收敛在回复适配器：**LingShuModelReplyAdapting** 负责最终文本归一（剥离 **M3** 等内联 **<think>**），**LingShuReplyStreamParsing** 负责流式增量归一（思考增量/正文增量两路，处理跨 **chunk** 标签切碎）。**State** 与 **View** 只消费归一化事件；新增模型族只在 **LingShuModelReplyAdapters** 登记，不改路由/执行/展示。

Views

路径：Sources/Views/

只放 **SwiftUI** 展示和交互。**Views** 可以读取 **LingShuState**，但不得直接调用 **CodexBridge**、网络客户端、文件仓储或外部 agent。

规则：

1. 新页面按功能建文件，不允许继续堆回主文件。
2. 复杂组件超过约 300 行时必须继续拆分。
3. 视图文案必须呈现真实状态，避免“已就位”这类无信息量文案。
4. 聊天主界面只展示灵枢统一人格；agent 过程进入调用链或任务执行记录。
5. 灵枢需要用户在 2~4 个有限方案里做决定时，必须通过路由协议的 **choices** 结构化选项渲染成选择卡（**LingShuChoiceCard**），点选即提交；不得把选项铺成长文让用户手打。
6. 极简语音模式是独立的小浮窗形态（约 340×560，右下角置顶）：视频画面 + 两条真实电平音轨 + 挂断/静音，由 **LingShuWindowPlacement.applyMinimalVoiceWindow** 负责窗口变形与恢复；不得再占满标准主窗。
7. 任务执行记录详情必须带右侧产出文件面板：聚合本轮与续接历史的所有产物，本机文件提供预

览/打开/访达操作。

Support

路径: Sources/Support/

放通用扩展、颜色、格式化、小工具。不得放业务流程。

Tests

路径: Tests/

新增规则或调度行为必须配测试。优先测试纯函数和服务边界:

1. 意图判断和任务路由。
2. 记忆命中、压缩、冷备回溯。
3. agent 调度顺序和权限边界。
4. 模型网关错误映射、流式心跳、重连策略。
5. 任务执行记录和产物清单。

macOS 可能拒绝直接加载 SwiftPM 在 .build 下生成的 XCTest bundle。项目测试入口统一使用 Scripts/run-tests.sh: 脚本会先构建测试包, 再复制到临时目录、清理扩展属性、重签名并调用 XCTest。后续不要把 swift test 卡住误判为测试逻辑失败。

Memory Standard

灵枢采用两层记忆:

1. 主线程记忆: 用于判断当前消息是否延续已有主题、是否需要加载历史线程、是否需要创建子线程。
2. 执行记忆: 用于恢复某个任务线程的执行过程、产物、错误、验收结果和下一步建议。

记忆过长时必须摘要压缩。长期未访问的记忆进入冷备库。冷备可以被检索, 但不得默认全量加载。

两层记忆之下是内嵌语义库 LingShuSemanticMemoryStore (SQLite + FTS5 + 本地句向量, 文件在 ~/Library/Application Support/LingShu/Memory/semantic-memory.sqlite3): 主线程/任务记忆写入时同步向量化入库, 检索时 FTS (中文按字 bigram 切词) 与向量余弦做 RRF 融合, 命中以「语义记忆」来源并入冷备通道。语义库是检索增强层, 不替代两层记忆的判断职责; 禁止引入外部数据库进程 (Redis 等), 单机记忆底座统一走 SQLite。

上下文预算由 LingShuContextCompressionEngine 执行: 超预算的旧轮次折叠成滚动摘要随上下文注入 (user 角色注入, 不得用 system 角色顶掉网关主提示), 折叠边界按块推进以保持前缀稳定; 禁止直接丢弃旧轮次。

任务回溯走三层语义匹配 (taskMemoryLookup), 不再是纯关键字交集: ①标签精确层 (≥ 2 个实义标签或线程指纹命中=高置信) ②语义检索层 (语义库 RRF, 全文命中为词面锚点, 纯向量单路因本地向量模型对无关中文短句有幽灵相似度而不单独触发确认) ③融合定置信度。规则:

1. 高置信直接续接; 中置信仅当用户措辞确实像在续接时 (isExplicitResumeRequest: 显式回溯动词 + 历史指代) 才发选择卡二次确认 ("继续这个 / 这是新任务"), 不赌错任务。措辞完整的全新任务即便与历史语义相近也直接按新任务推进, 不被 "继续还是新建" 打断——既避免每个沾边任务都被拦一道, 也避免非交互场景 (测试 / 定时触发 / 无头 MCP 驱动) 因无人应答选择卡而永久挂起。

2. 用户明确点名回溯 (`isExplicitResumeRequest`) 但未精确命中时, 给 top3 语义近似候选 + "都不是, 按新任务开始", 由用户挑选。
3. 续接靠 `task:<taskID>` 标签从语义条目回链到任务线程——`rememberTask` 写语义库时必须携带该标签。
4. 选择卡的结构化动作走 `CodexRouteChoiceOption.action(resume:<taskID>/new-task)`, 与模型生成的普通选项 (无 `action`, `label` 作输入提交) 共用一套渲染。

Clarification / Decision Standard

所有"需要人给一个决策"的场景 (任务续接确认、意图澄清、高风险操作批准、自主模式缺口确认) 都是同一个原语, 必须共用统一的决策编排中心 `LingShuClarificationCenter`, 不得再各写一套单槽状态互相覆盖。

规则:

1. 统一队列: 待确认问题进 `LingShuClarificationCenter` 的有序队列, 多个问题逐题呈现 (多轮弹窗) ——前一个答完后一个自动浮现, 绝不互相覆盖、绝不丢失。任何新的确认场景都走 `submit(kind:taskRecordID:present:autoResolve:)` 接入, 呈现与消解逻辑由调用方以闭包注入 (中心不绑定具体问题类型)。
2. 先消解再问: 能从默认假设 / 记忆 / 情境自动消解的, 调用方不要入队 (保持"默认落地交付、不过度反问")。只有真正消解不了的残差才提交给用户。
3. 非交互兜底: 自主运行 (非观察模式) / 无头驱动 / 定时触发等 `isNonInteractive` 为真的场景, 新问题直接走 `autoResolve` 安全默认, 不入队、不弹卡、不阻塞无人值守执行。这是任务续接死锁那一类 (单槽问题无人应答→任务永久挂起) 的根治。
4. 多轮推进: 用户点选 / 文字答复后, 消解路径必须调用 `advanceAfterExternalResolution()` (或 `resolveActive`) 出队当前题并呈现下一题; 消解过程中新产生的问题自然排到队尾。
5. 可审计: 每个问题的呈现、消解方式 (用户答复 / 自动默认) 都进任务执行记录或调用链。

Task Journal Standard

每条用户消息必须创建任务执行记录。记录内容以“群聊式进度”呈现, 包括:

1. 用户原始请求。
2. 灵枢主线程判断。
3. 记忆检索结果。
4. 模型网关状态。
5. 被调用 `agent` 的发言式进度。
6. 产出物清单。
7. `Review` 和交付结论。

如果新任务续接历史任务, 执行记录必须能关联并展示历史任务链路。

Chat History Standard

聊天气泡是用户感知灵枢连续性的第一入口, 必须独立持久化, 不能只依赖任务执行记录或主线程摘要。

规则：

1. **chatMessages** 的新增、更新和删除必须进入聊天历史存储。
2. 默认热加载最近 3 天的聊天历史，保证 App 重启后对话连续。
3. 超过热加载窗口的消息进入冷备库，不在启动时全量加载。
4. 用户向上翻历史时，界面应自动分页拉取冷备消息；冷备加载流程对用户透明。
5. 加载中的临时气泡不得持久化为正式历史，避免重启后出现永远转圈的回复。
6. 语音输入、视觉触发、插件消息和文字输入最终都必须落成同一种 **ChatMessage**，再复用统一存储链路。

默认文件位置：`~/Library/Application Support/LingShu/History/chat-hot.json` 和 `~/Library/Application Support/LingShu/History/chat-cold.json`。后续如果历史量继续增长，可以在不改变界面调用方式的前提下迁移到 SQLite。

Design Delivery Standard

设计部是通用设计交付能力，不是 PPT 专用脚本。它负责把用户目标转成受众判断、叙事结构、视觉方向、页级版式、图表建议和交付验收口径。

当用户提出 PPT、演示文稿、汇报材料、版式、视觉方案、设计稿等请求时，灵枢必须先走主线程判断，再按需创建设计交付任务。复杂交付必须至少经过规划、审议、调度和设计节点；如果需要真实文件，应继续调度执行节点生成文件，并由验证节点检查可打开性、页数、版式完整性和文件路径。

设计部不得直接耦合某个幻灯片库、模板或模型供应商。PPTX、HTML 演示页、图片素材、图表和外部设计 agent 都应通过执行器或外部 agent 网关接入，并把产物登记到任务执行记录的产出物清单。

Local Artifact Standard

灵枢应优先在本机完成常用文件的组装、落盘和预览。文本、Markdown、PDF、PPTX、HTML、JSON、CSV 等文件由 **Services** 层的产出物构建器生成；模型或 agent 可以提供内容和结构建议，但文件打包、路径登记和可打开性检查不应依赖云端文件服务。

规则：

1. 每类产出物优先使用独立 **builder**，避免把文档、代码、演示、媒体逻辑堆回主线程。
2. 生成后的真实文件必须进入任务执行记录的产出物清单，不能只在聊天气泡里描述“已生成”。
3. 图片、视频、音频等媒体预览必须优先使用本机 AppKit、AVKit、QuickLook 或 WebKit，不得默认上传到云端预览。
4. 云端模型可以作为内容生成、视觉理解、语音理解或设计建议的 **provider**，但不能成为本地文件生成与预览的唯一依赖。
5. 新增 Word、Excel、LaTeX、图表、设计稿、视频剪辑等格式时，仍按“能力判断 -> builder/执行器 -> 产物清单 -> 本地预览/系统打开”的路径接入。

Expert Collaboration Standard

任务执行走协同管线（`LingShuState+Collaboration.swift`），不是“每个 agent 说一句台词”的剧场：

1. 专家必须有档案（`LingShuExpertProfileRegistry`，协议化可插拔）：知识要点（专

- 业底线) + 交付物模板 (格式一致性) + 评审清单。内置项目经理 (项目分析)、产品经理 (PRD)、架构师 (架构文档)、设计总监、工程执行五类与评审官。
2. 管线五阶段都是真实模型调用: 规划 (计划+验收标准) → 专家产出 (按模板, 流式上屏) → 评审-纠正循环 (逐条核对, 结论只有通过/需修正; 被打回则按意见修订并复审, 最多三轮, 轮次用尽仍未通过要如实标注) → 验收 (最终结论+下一步建议)。全程进任务执行记录。
 3. 任务上下文严格隔离: 管线调用不携带聊天历史; 同任务多次迭代通过执行记录血缘继承前序结论, 并按队列严格顺序执行; 不同任务互不可见。
 4. 新增专家只在档案注册表登记; 接外部知识库/专家库时实现 **LingShuExpertProfileProviding** 整体替换, 编排流程不感知来源。
 5. 专家产出阶段具备真实工具执行 (**LingShuToolExecutor**, 协议化可换沙箱实现):
`read_file / write_file / list_directory / fetch_url / run_command`, 经「【工具】」行协议由宿主代为执行并回传结果 (agentic 循环, 最多 4 回合)。写文件限定工作目录内; 敏感位置 (.ssh/钥匙串/身份档案等) 拒绝读取; `run_command` 受「高风险动作需人工确认」策略约束且有危险命令黑名单; 所有工具调用与结果进任务执行记录可审计, 写盘成果自动登记产物清单。
 6. 任务级文件隔离: 每个管线在 工作目录/`tasks/<id>/` 独立子目录落盘, 并行任务物理隔离不互相污染。
 7. 评审走 **checklist** 驱动 (**LingShuChecklistVerdict**): 评审官按验收标准逐条   核对, 全部通过且明确判过才算过 (嘴上说通过但有  行不算, 防"假完成"); 不达标打回修订并复核, 最多三轮。
 8. 经验规则复利 (**LingShuMemoryService** 经验规则): 任务被打回又修正后, 把「问题→修正」提炼成一句通用规则写入语义库 (`kind=经验规则`), 下次同领域任务的规划阶段自动召回注入——记忆从流水账变成复利。
 9. 专家档案可由用户扩展: `~/Library/Application Support/LingShu/Skills/*.md` (frontmatter title/triggers + 小节) 经 **LingShuSkillLoader** 加载, **LingShuCompositeExpertRegistry** 与内置专家合并, 触发词命中时优先; 评审官固定内置不可覆盖。
 10. 外部工具经 MCP 连接器接入 (**LingShuMCPConnector/LingShuConnectorRegistry**, stdio JSON-RPC):
发现的工具拼进工具目录, `mcp:<tool>` 调用路由到对应 server。配置存 `~/Library/Application Support/LingShu/Connectors/servers.json`。

Residency & Trigger Standard

1. 灵枢常驻: 关闭主窗口不退出, 菜单栏值守 (状态/后台任务/定时任务一览, 可拉回主窗或退出); 开机自启经 **SMAppService** 由用户在配置页开关。
2. 定时触发 (**LingShuScheduledTriggerService**, 本机 JSON 持久化): 到点的指令以插件来源进入主线程正常判断——内容是提醒就由直答通道开口, 是任务就走完整协同管线 (例行任务)。一次性任务触发后自动停用, 同一分钟防重复触发。
3. 主动性边界: 灵枢不做无来由的主动插话; 任务进度在用户问询时如实报告 (情境上下文带阶段与耗时), 任务完成时主动发结构化汇报, 定时触发按用户登记的计划开口。

Agent Standard

灵枢能力节点必须是通用能力，不允许写死为某个一次性任务。推荐能力类型：

1. 规划
2. 执行
3. 监控
4. 验证
5. 记忆
6. 安全
7. 知识
8. 设计交付
9. 语音交互
10. 视觉解析
11. 实时感知路由

外部 agent 接入必须通过注册表和网关，不得在主线程中硬编码远端地址和协议细节。

File Size Guideline

1. App 入口文件目标小于 200 行。
2. 单个 SwiftUI 页面目标小于 300 行。
3. 单个服务文件目标小于 500 行。
4. 超过 800 行必须拆分，除非是明确的数据目录或生成代码。
5. LingShuState 只是过渡性聚合点，后续应继续拆成更小的 store 和 coordinator。

Review Checklist

每次迭代完成前检查：

1. 主文件是否仍只承担启动职责。
2. UI 是否直接调用底层模型或系统命令。
3. 新能力是否有明确层级归属。
4. 普通问答和复杂任务是否都有任务记录。
5. agent 状态是否只展示当前任务真实参与者。
6. 模型断线、重连、心跳是否有用户可理解状态。
7. 构建和核心测试是否通过。